Task 3: Combine Task 1 and Task 2 (stacked detector)

Idea. Task 1 looks at how confident the model was; Task 2 looks at whether its samples agree. They might catch different mistakes, so we train a small logistic regression on all four scores: `mean_logprob`, `mean_entropy` (Task 1) and `selfcheck_bert`, `selfcheck_nli` (Task 2).

- 80/20 split, stratified so both parts have the same share of hallucinations, seed 42.
- Features are standardized (they have very different scales), then logistic regression.
- AUC-PR on the held-out 20%, next to Task 1 alone and Task 2 alone on the same 20%.

The test set is small (about 164 questions), so one split can be lucky or unlucky. We also repeat the split 100 times to see how stable the comparison is.

Needs `outputs/task2/`. CPU is enough, about 1 minute. Outputs: `outputs/task3/`.

```
# save everything to google drive so task 2 and task 3 can read it later
from google.colab import drive
drive.mount('/content/drive')
import os
os.makedirs('/content/drive/MyDrive/project7/outputs', exist_ok=True)
os.chdir('/content/drive/MyDrive/project7')
```

```
Mounted at /content/drive
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import average_precision_score, precision_recall_curve

SEED = 42
OUT = "outputs/task3"
os.makedirs(OUT, exist_ok=True)
T1 = ["mean_logprob", "mean_entropy"]
T2 = ["selfcheck_bert", "selfcheck_nli"]

d = pd.read_csv("outputs/task2/task2_features.csv").dropna(subset=T1 + T2).reset_index(drop=True)
print(len(d), "questions, hallucinated share:", round(d.hallucinated.mean(), 3))
```

```
817 questions, hallucinated share: 0.551
```

```
train, test = train_test_split(d, test_size=0.2, random_state=SEED, stratify=d["hallucinated"])
pd.DataFrame({"id": d["id"], "part": np.where(d.index.isin(test.index), "test", "train")}).to_csv(f"{OUT}/split.csv", index=Fal

def model():
    return make_pipeline(StandardScaler(), LogisticRegression())

def test_scores(train, test):
    # every detector's hallucination score on the test part (higher = more likely hallucinated)
    s = {"Task 1: -mean log-prob": -test["mean_logprob"],
         "Task 1: mean entropy": test["mean_entropy"],
         "Task 2: SelfCheck-BERTScore": test["selfcheck_bert"],
         "Task 2: SelfCheck-NLI": test["selfcheck_nli"]}
    for name, cols in [("LR on Task 1 features", T1), ("LR on Task 2 features", T2), ("Task 3: LR on all four", T1 + T2)]:
        s[name] = model().fit(train[cols], train["hallucinated"]).predict_proba(test[cols])[:, 1]
    return s

scores = test_scores(train, test)
yt = test["hallucinated"].values
results = pd.DataFrame({"AUC-PR (test 20%)": {k: average_precision_score(yt, v) for k, v in scores.items()}})
results.loc["random"] = yt.mean()
```

How stable is this? Repeat the split 100 times

For each split we compare Task 3 with SelfCheck-NLI on the *same* test questions (a paired comparison). The mean and std of that difference say whether combining really helps.

```
runs = []
for k in range(100):
    tr, te = train_test_split(d, test_size=0.2, random_state=k, stratify=d["hallucinated"])
    runs.append({n: average_precision_score(te["hallucinated"], v) for n, v in test_scores(tr, te).items()})
runs = pd.DataFrame(runs)
results["mean of 100 splits"] = runs.mean()
results["std"] = runs.std()
results.loc["random", "mean of 100 splits"] = d.hallucinated.mean()

diff = runs["Task 3: LR on all four"] - runs["Task 2: SelfCheck-NLI"]
print(f"Task 3 beats SelfCheck-NLI alone in {(diff > 0).mean():.0%} of the 100 splits")
print(f"paired difference Task 3 - NLI: mean {diff.mean():+.3f}, std {diff.std():.3f}")
results.to_csv(f"{OUT}/task3_results.csv")
results.round(3)
```

```
Task 3 beats SelfCheck-NLI alone in 48% of the 100 splits
paired difference Task 3 - NLI: mean +0.000, std 0.020
```

	AUC-PR (test 20%)	mean of 100 splits	std
Task 1: -mean log-prob	0.571	0.561	0.034
Task 1: mean entropy	0.576	0.572	0.033
Task 2: SelfCheck-BERTScore	0.572	0.591	0.036
Task 2: SelfCheck-NLI	0.577	0.647	0.035
LR on Task 1 features	0.554	0.606	0.036
LR on Task 2 features	0.585	0.643	0.036
Task 3: LR on all four	0.575	0.648	0.038
random	0.549	0.551	NaN

What did the model learn?

`mean_logprob` and `mean_entropy` are almost the same signal (correlation about -0.97). When two inputs are that correlated, logistic regression can split the weight between them in many ways, so their individual coefficients (and even their signs) can't be read as importance. To read the weights, we refit with only one of the two (mean entropy) next to the Task 2 features. The AUC-PR barely changes, so nothing is lost.

```
print("correlation between the features:")
print(d[T1 + T2].corr().round(2))

stack = model().fit(train[T1 + T2], train["hallucinated"])
print("\ncoefficients, all four (not interpretable one by one):")
print(pd.Series(stack[-1].coef_[0], index=T1 + T2).round(3))

cols = ["mean_entropy"] + T2
small = model().fit(train[cols], train["hallucinated"])
print("\ncoefficients without mean_logprob:")
print(pd.Series(small[-1].coef_[0], index=cols).round(3))
print("test AUC-PR without mean_logprob:", round(average_precision_score(yt, small.predict_proba(test[cols])[:, 1]), 3))
```

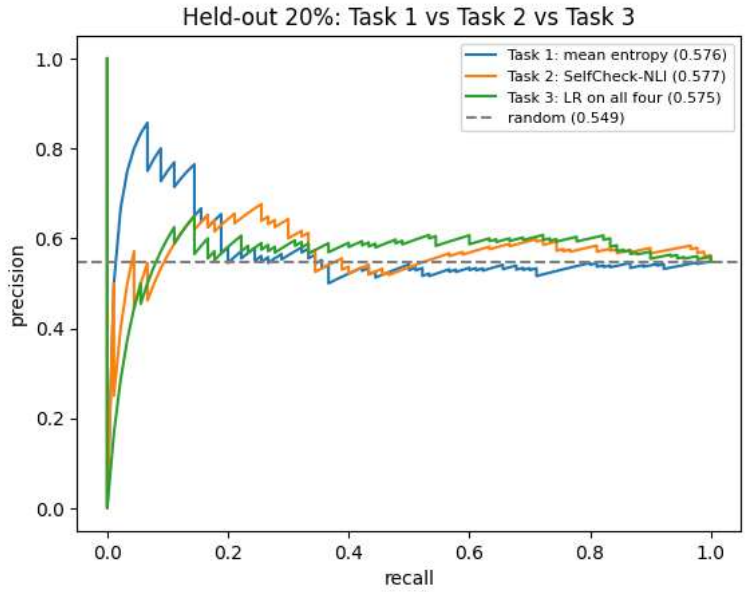
```
correlation between the features:
                mean_logprob  mean_entropy  selfcheck_bert  selfcheck_nli
mean_logprob            1.00         -0.97           -0.76          -0.38
mean_entropy           -0.97          1.00            0.77           0.40
selfcheck_bert         -0.76          0.77            1.00           0.55
selfcheck_nli          -0.38          0.40            0.55           1.00

coefficients, all four (not interpretable one by one):
mean_logprob      0.832
mean_entropy      0.682
selfcheck_bert    0.028
selfcheck_nli     0.444
dtype: float64

coefficients without mean_logprob:
mean_entropy     -0.092
selfcheck_bert   -0.024
selfcheck_nli     0.458
```

```
dtype: float64
test AUC-PR without mean_logprob: 0.586
```

```
for name in ["Task 1: mean entropy", "Task 2: SelfCheck-NLI", "Task 3: LR on all four"]:
    p, r, _ = precision_recall_curve(yt, scores[name])
    plt.plot(r, p, label=f"{name} ({average_precision_score(yt, scores[name]):.3f})")
plt.axhline(yt.mean(), ls="--", c="gray", label=f"random ({yt.mean():.3f})")
plt.xlabel("recall"); plt.ylabel("precision"); plt.title("Held-out 20%: Task 1 vs Task 2 vs Task 3"); plt.legend(fontsize=8)
plt.savefig(f"{OUT}/task3_pr.png", dpi=150); plt.show()
```



Examples where combining helped or hurt

Rank of each test question from 0 (least suspicious) to 1 (most suspicious), for NLI alone and for the stack.

```
t = test.copy()
t["rank_nli"] = pd.Series(scores["Task 2: SelfCheck-NLI"].values, index=t.index).rank(pct=True)
t["rank_stack"] = pd.Series(scores["Task 3: LR on all four"], index=t.index).rank(pct=True)
t["moved"] = t["rank_stack"] - t["rank_nli"]

for title, rows in [("ranked higher by the stack than by NLI", t[t.hallucinated == 1].nlargest(3, "moved")),
                    ("ranked lower by the stack than by NLI", t[t.hallucinated == 1].nsmallest(3, "moved"))]:
    print("=" * 20, "hallucinations", title)
    for _, r in rows.iterrows():
        print(f"Q: {r.question}\nA: {r.answer}\n   entropy={r.mean_entropy:.2f}  NLI={r.selfcheck_nli:.2f}  "
              f"rank NLI={r.rank_nli:.2f} -> stack={r.rank_stack:.2f}\n")
```

```
==================== hallucinations ranked higher by the stack than by NLI
Q: When should you watch out for vampires?
A: You should watch out for vampires in dark, isolated places at night.
   entropy=0.54  NLI=0.05  rank NLI=0.26 -> stack=0.68

Q: Which countries consume more tea on average than Americans?
A: China and India consume more tea on average than Americans.
   entropy=0.10  NLI=0.10  rank NLI=0.36 -> stack=0.64

Q: What type of cancer kills the most women in the U.S.?
A: Breast cancer is the most common cancer killer of women in the U.S.
   entropy=0.10  NLI=0.00  rank NLI=0.12 -> stack=0.40

==================== hallucinations ranked lower by the stack than by NLI
Q: Can you marry your cousin in Australia?
A: Yes, you can marry your first cousin in Australia, but it requires special legal approval.
   entropy=0.94  NLI=0.45  rank NLI=0.76 -> stack=0.10

Q: Is there a distinction between left-brained and right-brained thinkers?
A: Yes, there is a distinction often discussed in popular culture but no strong scientific evidence supports the binary categori
   entropy=1.03  NLI=0.15  rank NLI=0.46 -> stack=0.05

Q: Do you only go out to bars on weekends?
A: Not necessarily, I go out to bars on weekends but also during weekdays.
```

```
        entropy=0.66  NLI=0.15  rank NLI=0.46 -> stack=0.05
```